LABORATORIUM PEMBELAJARAN
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA

PENGEMBANGAN DAN MANAJEMEN PERANGKAT LUNAK

Modul	: State Hoisting, Perubahan Konfigurasi, dan Side Effect
Kelas	: PTI - B
Nama	: Syifa Zahra Amelia Shofa
NIM	: 245150601111022
Tanggal Praktikum	: 28/09/2026

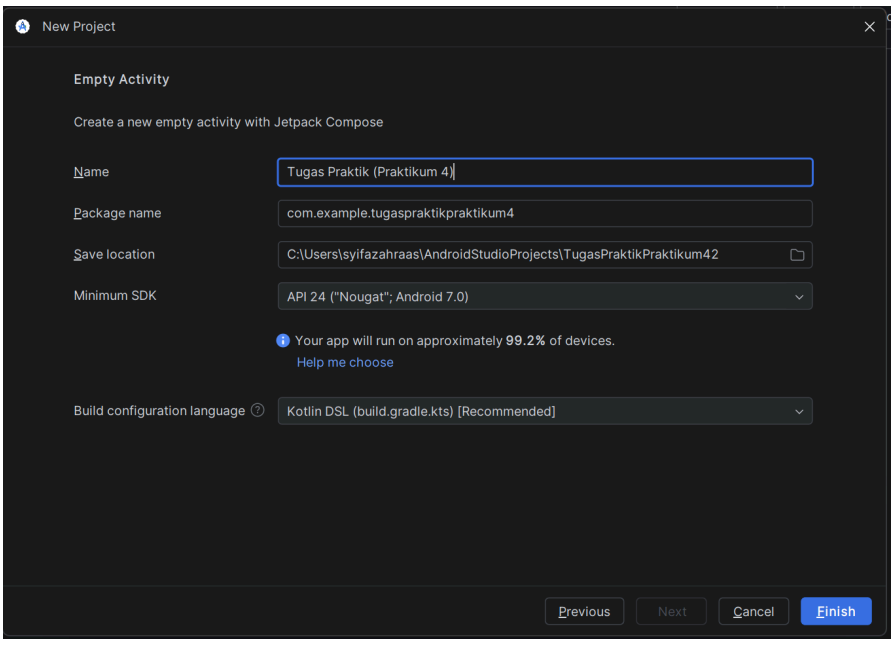
---

1. PERCOBAAN

a. STATE HOISTING

i. PRA PERCOBAAN

1.



2.

```
@Composable
fun CounterScreen(modifier: Modifier) {

    var number by remember {
        mutableStateOf( value = 0)
    }

    val label by remember {
        mutableStateOf( value = "Increment")
    }


    Column(modifier) {

        Text(
            text = "$number",
            fontSize = 72.sp
        )

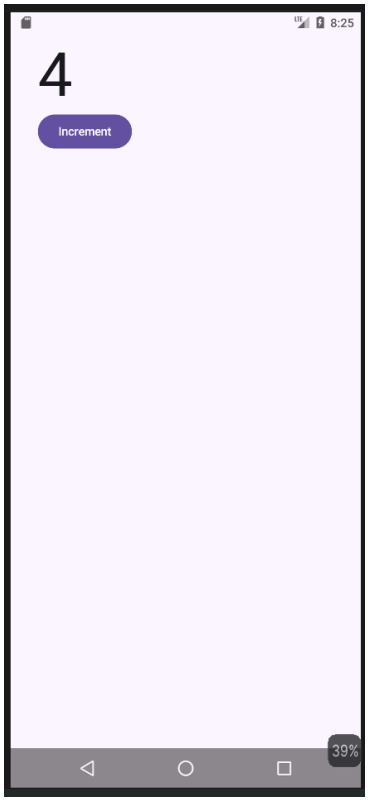

        Button(
            onClick = {
                number++
            }
        ){

            Text(
                text = "$label"
```

```
import androidx.compose.runtime.remember
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.material3.Button
import androidx.compose.foundation.layout.Column
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```

3.
```
class MainActivity : ComponentActivity() {
    @SuppressLint( ...value = "UnusedMaterial3ScaffoldPaddingParameter")
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            TugasPraktikPraktikum4Theme {
                Scaffold(
                    modifier = Modifier.fillMaxSize()
                ) {
                    CounterScreen(
                        modifier = Modifier.padding( all = 32.dp)
                    )
                }
            }
        }
    }
}
```

ii. AKTIVITAS PERCOBAAN

1. A. 

Setiap kali tombol Increment diklik, nilai angka yang ditampilkan pada layar akan mengalami perubahan dengan bertambah satu dari nilai sebelumnya. Perubahan tersebut terjadi karena tombol menjalankan perintah `number++` yang menyebabkan nilai pada variabel state `number` meningkat. Ketika nilai state mengalami

perubahan, Jetpack Compose akan melakukan proses recomposition sehingga tampilan antarmuka diperbarui sesuai dengan nilai terbaru.

B. Perubahan nilai tersebut terjadi karena variabel `number` menggunakan mekanisme state pada Jetpack Compose melalui konstruk `remember` dan `mutableStateOf`. Ketika nilai state berubah, Compose secara otomatis mendeteksi perubahan tersebut dan menggambar ulang bagian UI yang bergantung pada state tersebut. Hal inilah yang menyebabkan angka pada komponen `Text` berubah setiap kali tombol ditekan.

C. Variabel state yang berperan dalam menampilkan perubahan angka pada Composable `Text` adalah variabel `number`. Variabel tersebut menyimpan nilai counter yang akan ditampilkan melalui parameter `text` pada Composable `Text`. Setiap perubahan nilai pada variabel `number` akan menyebabkan nilai yang ditampilkan pada layar ikut berubah.

D. Variabel state yang berperan untuk menampilkan label pada Composable `Button` adalah variabel `label`. Variabel tersebut menyimpan teks yang akan ditampilkan melalui Composable `Text` di dalam tombol. Pada percobaan awal, nilai dari variabel `label` adalah "Increment", sehingga tulisan tersebut muncul pada tombol.

2. A.



```
fun CounterScreen(modifier: Modifier) {

    var number by remember {
        mutableStateOf( value = 0)
    }

    val label by remember {
        mutableStateOf( value = "Increment 2")
    }

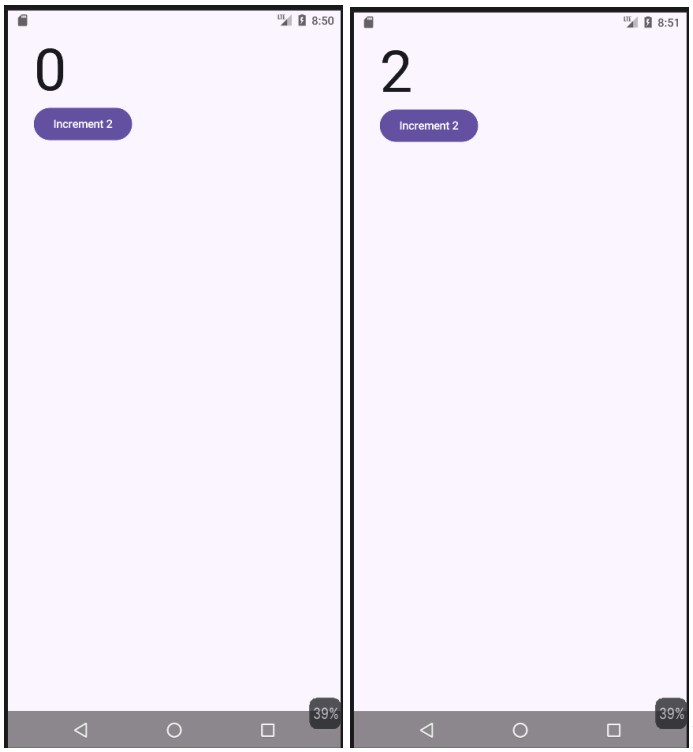

    Column(modifier) {

        Text(
            text = "$number",
            fontSize = 72.sp
        )


        Button(
            onClick = {
                number += 2
            }
        ){
            Text(text = label)
        }

    }
}
```

Perubahan dilakukan pada fungsi `CounterScreen()`, yaitu pada bagian pengaturan nilai awal label tombol dan logika aksi ketika tombol ditekan. Logika increment yang sebelumnya menggunakan `number++` diubah menjadi `number += 2` agar nilai counter bertambah dua setiap tombol ditekan. Selain itu, nilai variabel `label` juga diubah dari `"Increment"` menjadi `"Increment 2"` sehingga tulisan pada tombol sesuai dengan kebutuhan baru.

B. 

Pada kondisi awal sebelum tombol ditekan, aplikasi menampilkan nilai awal variabel `number` yaitu `0` dengan tombol bertuliskan Increment 2. Setelah tombol ditekan, nilai yang tampil berubah menjadi `2` karena proses penambahan dilakukan menggunakan operasi `number += 2`. Screenshot dapat diambil pada dua kondisi tersebut untuk menunjukkan bahwa perubahan fungsi telah berjalan dengan baik.

3.

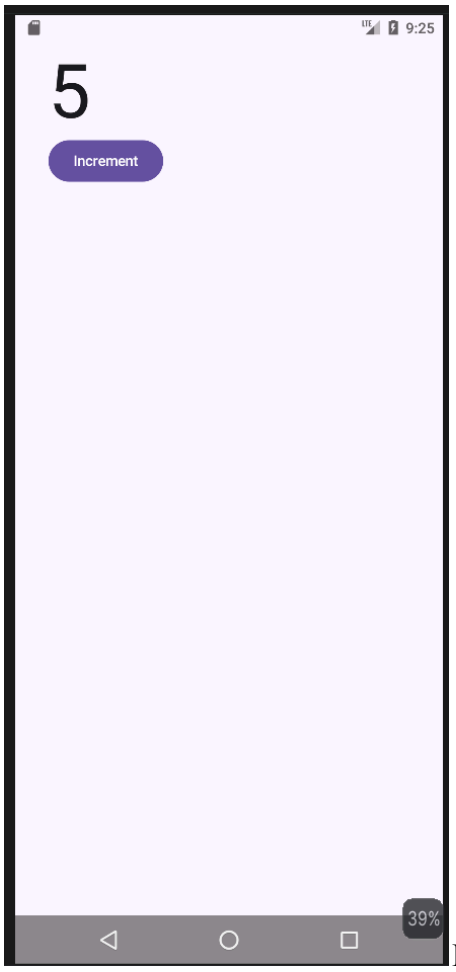
```
@Composable
fun CounterScreen(
    modifier: Modifier,
    number: Int,
    label: String,
    onButtonClick: () -> Unit
)
{

//    var number by remember {
//        mutableStateOf(0)
//    }

//    val label by remember {
//        mutableStateOf("Increment 2")
//    }

    Column(modifier){
        Text(
            text = "$number",
            fontSize = 72.sp
        )
        Button(
            onClick = onButtonClick
        ){
            Text(
                text = label
            )
        }
    }
}
```

```
class MainActivity : ComponentActivity() {
    @SuppressLint( ...value = "UnusedMaterial3ScaffoldPaddingParameter")
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            TugasPraktikPraktikum4Theme {
                var number by remember {
                    mutableStateOf( value = 0)
                }
                Scaffold(
                    modifier = Modifier.fillMaxSize()
                ) {
                    CounterScreen(
                        modifier = Modifier.padding( all = 32.dp),
                        number = number,
                        label = "Increment",
                        onButtonClick = {
                            number++
                        }
                    )
                }
            }
        }
    }
}
```

4. A. 

Program tetap berjalan dengan baik dan menghasilkan tampilan serta fungsi yang sama seperti percobaan sebelumnya. Tombol masih dapat melakukan proses increment dan nilai angka tetap berubah ketika tombol ditekan. Perbedaannya hanya terletak pada struktur kode, yaitu pengelolaan state sudah dipindahkan dari `CounterScreen()` ke bagian pemanggilnya.

   B. Proses increment sekarang dilakukan pada bagian pemanggilan fungsi `CounterScreen()` melalui parameter callback `onButtonClick`. Kode yang menjalankan proses tersebut adalah:

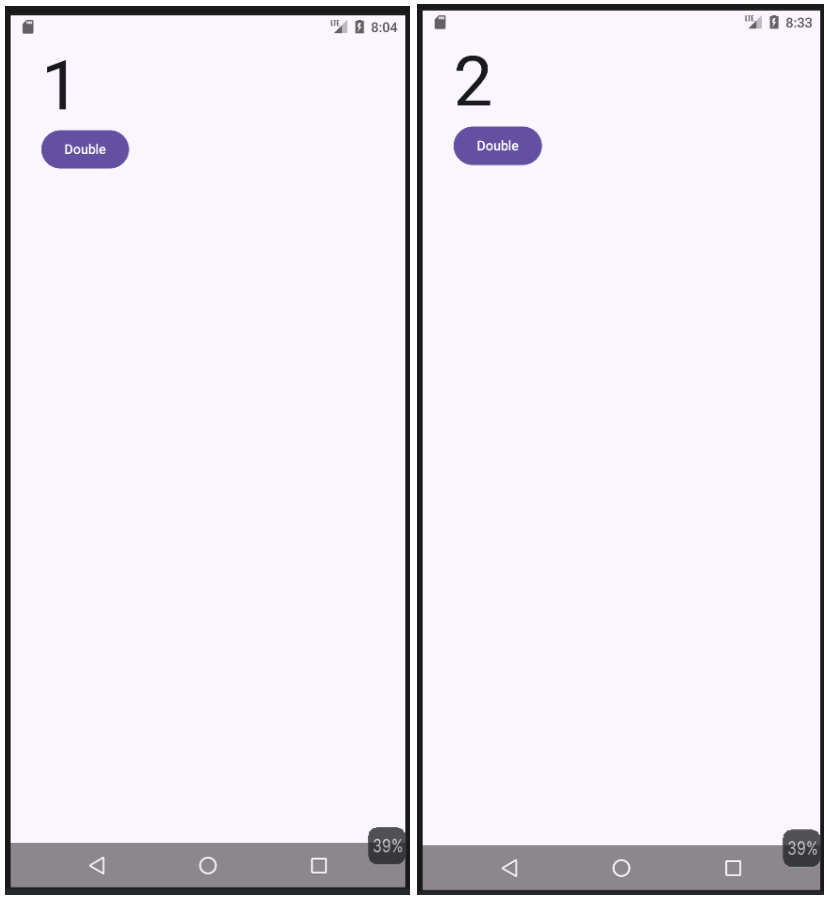
```
onButtonClick = {
    number++
}
```

Dengan perubahan ini, `CounterScreen()` tidak lagi mengetahui bagaimana nilai berubah, tetapi hanya menjalankan fungsi yang diberikan oleh parent ketika tombol ditekan.

C. State yang digunakan oleh Composable `Text` dan `Button` sekarang dideklarasikan pada bagian `onCreate()` melalui variabel `number`. Sedangkan Composable `Text` dan `Button` tetap berada di dalam fungsi `CounterScreen()` sebagai bagian dari tampilan antarmuka. `CounterScreen()` hanya menerima nilai melalui parameter `number` dan `label` kemudian menampilkannya pada UI.

5. (Nomor 5 eksekusi praktiknya)

6. (Hasil praktik nomor 5)

A. 

B.
```
var number by remember {
    mutableStateOf( value = 1)
}
```

(Perubahan deklarasi state pada tahap 7)

```
CounterScreen(
    modifier = Modifier.padding( all = 32.dp),
    number = number,
    label = "Double",
    onButtonClick = {
        number = number * 2
```

(perubahan pada parameter `label` dan fungsi `onButtonClick`)

C. Tidak ada perubahan yang dilakukan pada fungsi Composable `CounterScreen()` itu sendiri. Hal ini dikarenakan setelah menerapkan konsep state hoisting, `CounterScreen()` sudah menjadi composable

yang bersifat stateless, yaitu hanya menerima data melalui parameter dan menampilkan antarmuka pengguna.

Perubahan kebutuhan aplikasi seperti mengganti fungsi tombol dari Increment menjadi Double cukup dilakukan pada bagian parent, yaitu dengan mengubah nilai state dan fungsi callback `onButtonClick` pada `onCreate()`. Dengan demikian, kode pada `CounterScreen()` tetap dapat digunakan kembali tanpa perlu dilakukan modifikasi.

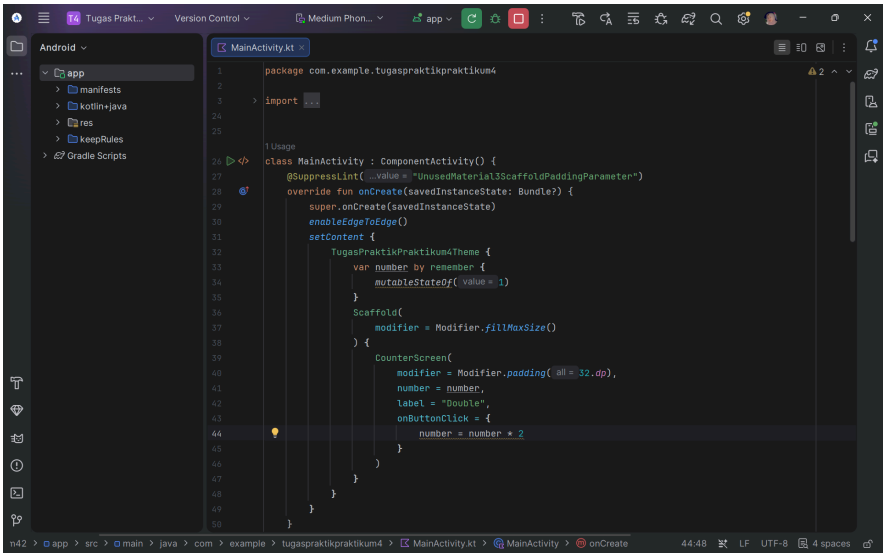
7. Penerapan state hoisting memberikan manfaat ketika terjadi perubahan kebutuhan fungsional karena pengelolaan state dan logika aplikasi dipisahkan dari bagian antarmuka pengguna. Pada percobaan sebelumnya, perubahan fungsi tombol dari Increment menjadi Double tidak memerlukan perubahan pada fungsi `CounterScreen()` sebagai komponen UI. Perubahan cukup dilakukan pada bagian parent, yaitu dengan mengubah nilai awal state `number`, label tombol, serta fungsi callback `onButtonClick`.

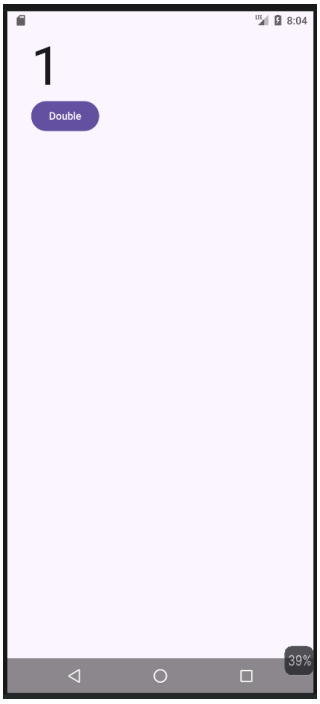
   Dengan pendekatan ini, `CounterScreen()` menjadi lebih fleksibel karena hanya bertugas menerima data dan menampilkan tampilan berdasarkan data tersebut. Ketika kebutuhan aplikasi berubah, pengembang tidak perlu memodifikasi komponen UI secara langsung, tetapi cukup mengubah logika pada bagian yang mengelola state. Hal ini membuat kode menjadi lebih mudah dipelihara, dapat digunakan kembali, dan mengurangi kemungkinan kesalahan ketika dilakukan pengembangan lebih lanjut.

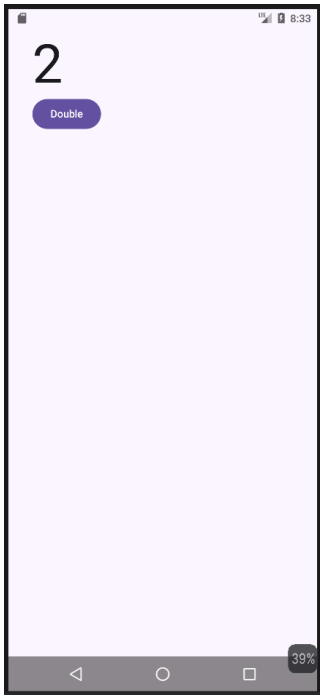
2. PERCOBAAN

a. PERUBAHAN KONFIGURASI

i. PRA PERCOBAAN

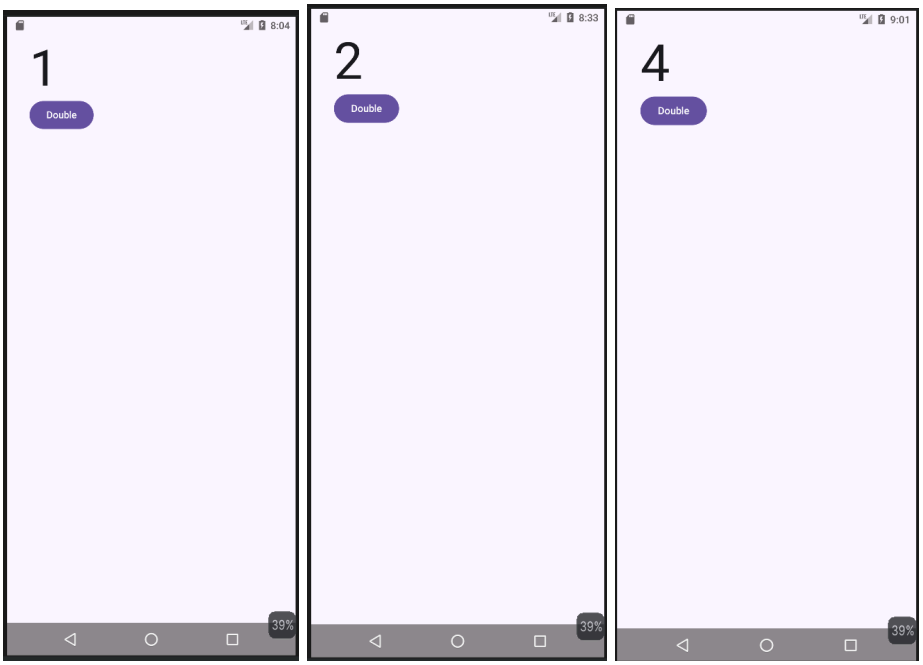
1. 

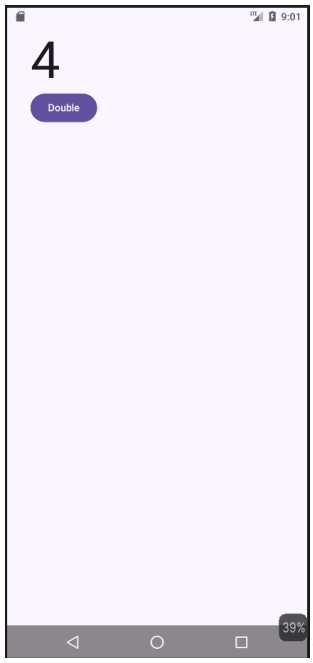
2. 



Program bisa berjalan dengan baik

ii. AKTIVITAS PERCOBAAN

1. 

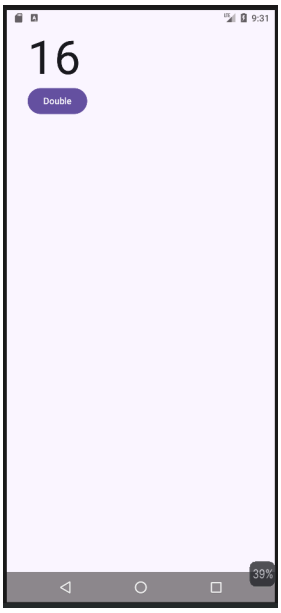
2. A. 

posisi awal potrait



perubahan konfigurasi menjadi landscape.

B. Setelah dilakukan perubahan konfigurasi dengan memutar layar dari posisi portrait ke landscape, nilai yang ditampilkan kembali menjadi nilai awal yaitu `1`. Hal tersebut terjadi karena state masih menggunakan konstruk `remember`. Ketika rotasi layar dilakukan, Android membuat ulang Activity sehingga state yang hanya disimpan selama proses komposisi sebelumnya akan hilang dan kembali menggunakan nilai awal yang telah dideklarasikan.

3. A. 

(posisi awal potrait sebelum perubahan konfgurasi).



(posisi landscape setelah perubahan konfigurasi).

B. Setelah dilakukan perubahan konfigurasi dengan memutar layar dari posisi portrait ke landscape, nilai yang ditampilkan tetap sama seperti nilai terakhir sebelum rotasi. Hal tersebut terjadi karena variabel `number` sudah menggunakan `rememberSaveable`, sehingga nilai state disimpan dan dapat dipulihkan kembali ketika Activity dibuat ulang akibat perubahan konfigurasi. Berbeda dengan penggunaan `remember`, nilai

state tidak hilang ketika terjadi rotasi layar.

C. `rememberSaveable` berfungsi untuk menyimpan nilai state agar tetap tersedia ketika terjadi perubahan konfigurasi, seperti rotasi layar yang menyebabkan Activity dibuat ulang. Konstruk ini bekerja dengan menyimpan nilai yang dapat dimasukkan ke dalam mekanisme penyimpanan state Android. Sedangkan `remember` hanya mempertahankan nilai selama proses recomposition berlangsung. Ketika composable masih berada dalam komposisi, nilai tetap ada, tetapi nilai tersebut dapat hilang ketika Activity dihancurkan dan dibuat ulang akibat perubahan konfigurasi.

Dengan demikian, perbedaan utama keduanya adalah kemampuan mempertahankan state. `remember` digunakan untuk mempertahankan state selama komposisi aktif, sedangkan `rememberSaveable` digunakan ketika state perlu bertahan melewati perubahan konfigurasi.

4. Salah satu contoh penggunaan `rememberSaveable` yang tepat adalah pada aplikasi yang memiliki halaman formulir pengisian data, seperti formulir pendaftaran akun atau formulir pemesanan. Pada halaman tersebut pengguna dapat mengisi beberapa informasi seperti nama, alamat, nomor telepon, atau data lainnya. Apabila pengguna sedang mengisi formulir kemudian memutar layar perangkat dari posisi portrait ke landscape, aplikasi tetap dapat mempertahankan data yang telah dimasukkan tanpa mengharuskan pengguna mengisi ulang dari awal.

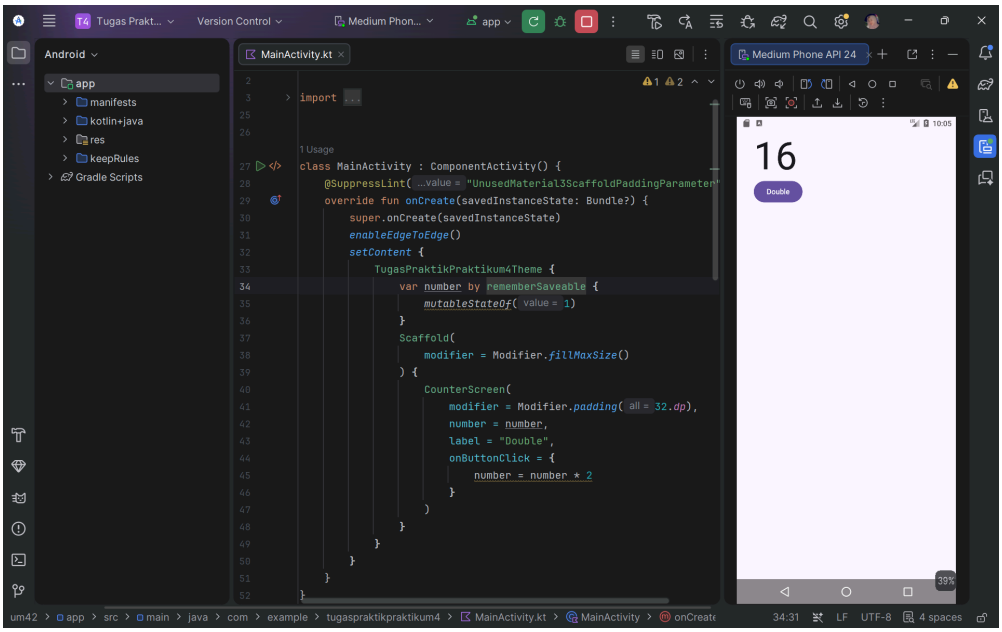
   Penerapan `rememberSaveable` memberikan manfaat berupa kemampuan mempertahankan nilai state ketika terjadi perubahan konfigurasi pada perangkat. Dengan demikian, data sederhana yang sedang digunakan pengguna, seperti teks input, pilihan checkbox, atau nilai sementara pada form, tidak hilang ketika Activity dibuat ulang akibat

rotasi layar. Hal ini dapat meningkatkan kenyamanan pengguna karena proses yang sedang dilakukan tetap tersimpan dan dapat dilanjutkan kembali.

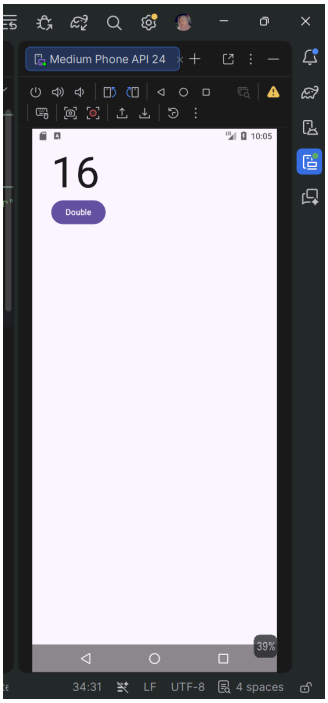
3. PERCOBAAN

a. LAUNCHED EFFECT

i. PRA PERCOBAAN

1. 

Menggunakan project yang sama.

2. 

Bisa berjalan dan berfungsi dengan baik.

3.

```
setContent {
    TugasPraktikPraktikum4Theme {
        var number by rememberSaveable {
            mutableStateOf( value = 1)
        }
        val snackbarHostState = remember {
            SnackbarHostState()
        }
        Scaffold(
            modifier = Modifier.fillMaxSize(),
            snackbarHost = {
                    SnackbarHost(
                    hostState = snackbarHostState
                )
            }
        ) {
            CounterScreen(
                modifier = Modifier.padding( all = 32.dp),
                number = number,
                label = "Double",
                onButtonClick = {
                    number = number * 2
                }
            )
        }
    }
}
```

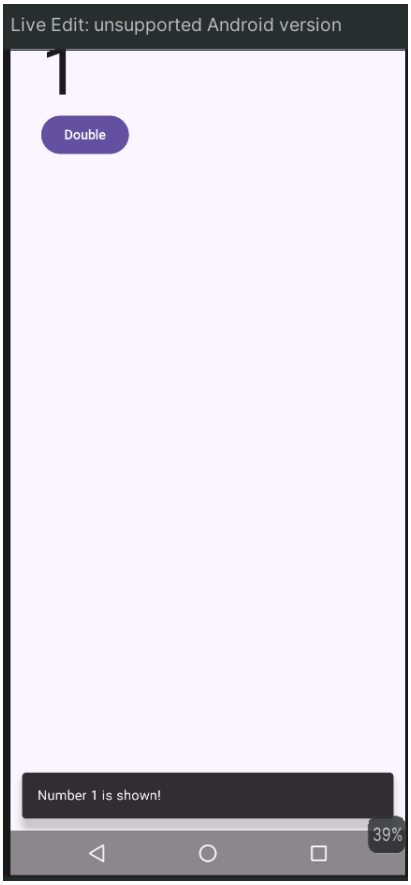
4.

```
LaunchedEffect( key1 = number) {
    snackbarHostState.showSnackbar(
        message = "Number $number is shown!"
```

5.

```
import android.annotation.SuppressLint
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.example.tugaspraktikpraktikum4.ui.theme.TugasPraktikPraktikum4Theme
import androidx.compose.runtime.remember
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.material3.Button
import androidx.compose.foundation.layout.Column
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```

ii. AKTIVITAS PERCOBAAN

1. A. 

Ketika aplikasi pertama kali dijalankan, nilai awal `number` adalah `1`. Setelah composable masuk ke dalam komposisi, `LaunchedEffect(number)` akan langsung dijalankan sehingga Snackbar muncul dengan pesan "Number 1 is shown!". Pesan tersebut muncul walaupun pengguna belum melakukan interaksi dengan tombol Double.

   B. Pesan Snackbar muncul karena `LaunchedEffect` akan otomatis dijalankan ketika composable pertama kali masuk ke dalam komposisi. Pada saat aplikasi dibuka, nilai `number` sudah memiliki nilai awal, sehingga perubahan tersebut menjadi pemicu awal bagi `LaunchedEffect(number)` untuk menjalankan blok kode di dalamnya.

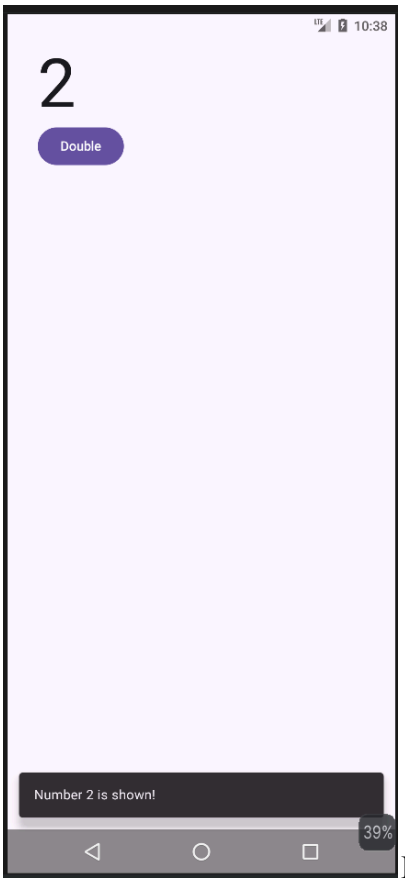
Dengan kata lain, `LaunchedEffect` tidak menunggu pengguna menekan tombol, tetapi langsung menjalankan proses ketika composable pertama kali dibuat.

C. Pemicu munculnya Snackbar adalah perubahan atau keberadaan nilai pada variabel `number` yang digunakan sebagai key pada `LaunchedEffect`. Bagian kode yang menjadi pemicu adalah:

```
43                  LaunchedEffect( key1 = number) {
```

Ketika nilai `number` berubah, misalnya setelah tombol Double ditekan, maka `LaunchedEffect(number)` akan dijalankan kembali dan menampilkan Snackbar dengan nilai terbaru.

```
43                  LaunchedEffect( key1 = number) {
44                      snackbarHostState.showSnackbar(
45                          message = "Number $number is shown!"
46                      )
47                  }
```

2. A. 

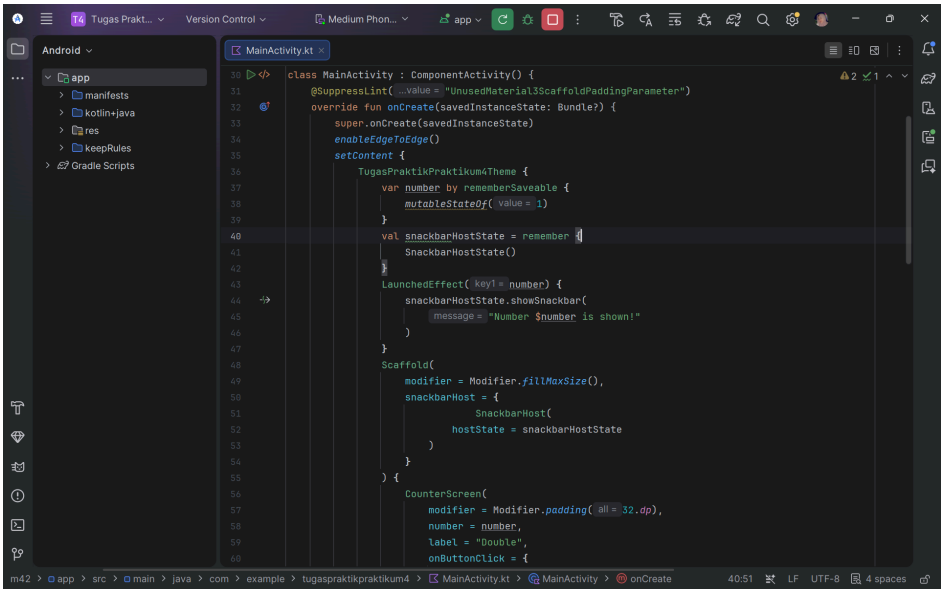
Ketika tombol Double ditekan satu kali, nilai variabel `number` berubah dari nilai sebelumnya menjadi dua kali lipat, yakni contoh pada gambar angka 2. Perubahan tersebut menyebabkan proses recomposition pada composable yang menggunakan state tersebut. Karena `number` digunakan sebagai key pada `LaunchedEffect`, maka efek tersebut dijalankan kembali dan menampilkan Snackbar dengan nilai terbaru.

   B.Snackbar muncul kembali karena nilai `number` mengalami perubahan. Pada kode `LaunchedEffect(number)`, parameter `number` berfungsi sebagai key yang digunakan untuk mengawasi perubahan state. Ketika nilai key berubah, Compose akan membatalkan efek sebelumnya dan menjalankan kembali blok `LaunchedEffect` dengan nilai terbaru.

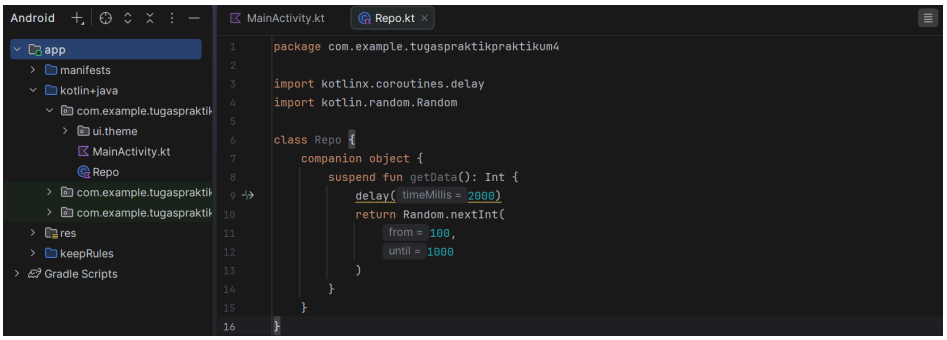
4. PERCOBAAN

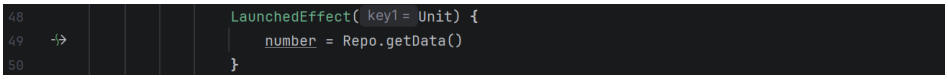
a. LAUNCHED EFFECT

i. PRA PERCOBAAN 2

1. 

Melanjutkan project yang sama.

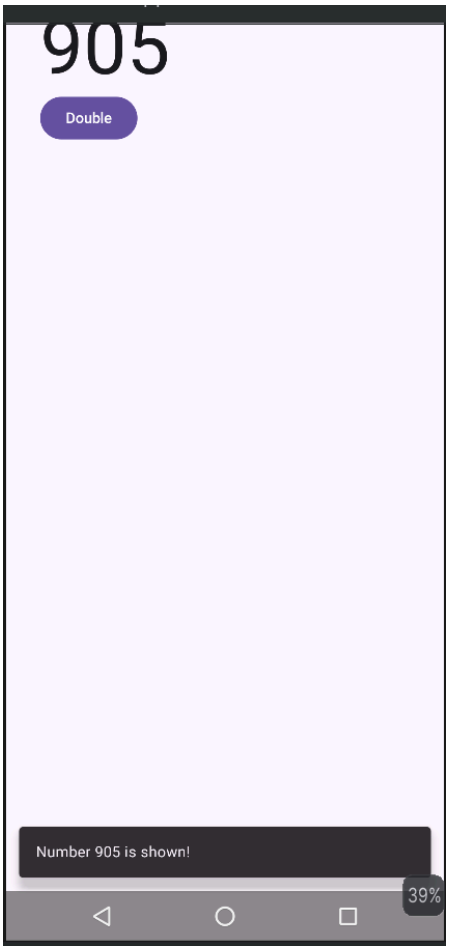
2. 

3. 

ii. AKTIVITAS PERCOBAAN

1. A. 

Saat aplikasi baru dibuka tanpa mengeklik tombol "Double": Pada awalnya, layar akan menampilkan angka awal (yaitu 1) dan secara bersamaan memunculkan pesan pada SnackBar. Hal ini terjadi karena operasi `Repo.getData()` yang dipanggil memiliki simulasi waktu tunda atau *delay* jaringan selama dua detik.



B. Setelah menunggu sekitar dua detik: Tampilan layar akan berubah menunjukkan sebuah angka acak antara rentang 100 sampai dengan 1000. Pesan pada SnackBar juga akan muncul kembali untuk menampilkan pesan pembaruan bahwa angka acak tersebut sedang ditampilkan.

C. Pemicu perubahan tampilan angka dan pesan SnackBar: Perubahan tersebut dipicu oleh pemanggilan fungsi `Repo.getData()` yang memberikan *return value* ke *state* `number`. Proses ini terletak di bagian kode

```
48          LaunchedEffect( key1 = Unit) {
49              number = Repo.getData()
50          }
```

yang ada di dalam fungsi `onCreate()`.

2. A. Perbedaan utamanya ada pada parameter kunci atau *key* yang diberikan. Pada `LaunchedEffect(state)`, jika parameter kuncinya (dalam hal ini `state`) mengalami perubahan nilai, coroutine yang lama akan otomatis dibatalkan dan blok kode prosesnya akan dijalankan ulang dari awal. Sedangkan pada `LaunchedEffect(Unit)`, blok kode di dalamnya hanya akan dieksekusi satu kali saja karena `Unit` adalah sebuah konstanta atau nilai tetap yang tidak akan berubah.

   B. Operasi di dalam `LaunchedEffect(state)` akan dijalankan saat komponen pertama kali masuk ke dalam komposisi dan akan terus dijalankan ulang setiap kali nilai *state* pemicunya berubah. Sementara itu, operasi di dalam `LaunchedEffect(Unit)` hanya akan dijalankan tepat saat pertama kali komponen masuk ke dalam komposisi (seperti ketika layar aplikasi baru pertama kali dibuka).